

Contents

1	Quantum Computing Fundamentals	2
1.1	Q1: What is a qubit and how does it differ from a classical bit?	2
1.2	Q2: Explain quantum entanglement and its applications	2
1.3	Q3: What is quantum superposition and how is it used?	3
1.4	Q4: Explain the measurement problem in quantum mechanics	3
1.5	Q5: What is quantum interference and how do algorithms exploit it?	4
2	Quantum Gates and Circuits	4
2.1	Q6: Describe fundamental single-qubit gates	4
2.2	Q7: Explain multi-qubit gates	5
2.3	Q8: How do you design and optimize quantum circuits?	6
2.4	Q9: Explain quantum circuit transpilation	7
2.5	Q10: What are limitations of NISQ devices?	8
3	Quantum Algorithms	10
3.1	Q11: Explain Grover's search algorithm in detail	10
3.2	Q12: Explain the Quantum Fourier Transform	12
3.3	Q13: Describe variational quantum algorithms (VQE, QAOA)	13

Introduction

This document contains exhaustive questions and answers for quantum developer interviews, covering fundamentals, algorithms, programming, hardware, and applications.

1 Quantum Computing Fundamentals

1.1 Q1: What is a qubit and how does it differ from a classical bit?

Complete Answer:

A qubit is the fundamental unit of quantum information with these key properties:

Mathematical Representation:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (1)$$

Key Differences from Classical Bits:

- **Superposition:** Exists in combination of 0 and 1 simultaneously
- **Measurement:** Collapses to 0 or 1 with probabilities $|\alpha|^2$ and $|\beta|^2$
- **Bloch Sphere:** Can be visualized as point on unit sphere with θ (polar) and ϕ (azimuthal) angles

Physical Implementations:

1. **Superconducting** (IBM, Google): Josephson junctions, fast gates (20ns), short coherence (100 μ s)
2. **Trapped Ions** (IonQ): Laser-controlled ions, slow gates (100 μ s), long coherence (seconds)
3. **Photonic** (Xanadu): Room temperature, no decoherence, hard 2-qubit gates
4. **Neutral Atoms** (QuEra): Flexible geometry, scalable to 1000+ qubits
5. **Silicon Spin** (Intel): CMOS compatible, atomic precision needed

Code Example (Qiskit):

```
from qiskit import QuantumCircuit
qc = QuantumCircuit(1, 1)
qc.h(0) # Create superposition: (|0> + |1>)/sqrt(2)
qc.measure(0, 0)
# Run multiple times to see probabilistic outcomes
```

1.2 Q2: Explain quantum entanglement and its applications

Complete Answer:

Entanglement creates correlations between qubits that cannot be explained classically.

Bell State Example:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (2)$$

Measuring one qubit instantly determines the other, regardless of distance.

Mathematical Property: Cannot be written as product state: $|\Phi^+\rangle \neq |\psi_A\rangle \otimes |\psi_B\rangle$

Applications:

1. **Quantum Teleportation:** Transfer quantum state using entanglement and classical communication
2. **Superdense Coding:** Send 2 classical bits using 1 qubit
3. **Quantum Cryptography:** E91 protocol for secure communication
4. **Error Correction:** Distribute quantum information across multiple qubits
5. **Algorithms:** Essential for quantum speedups (Shor's, Grover's)

Creating Entanglement:

```
qc = QuantumCircuit(2, 2)
qc.h(0) # Superposition on qubit 0
qc.cx(0, 1) # CNOT creates entanglement
qc.measure([0,1], [0,1])
# Results: Only |00> and |11> observed (never |01> or |10>)
```

Verification: Violates Bell inequality (CHSH): $S > 2$ (quantum) vs $S \leq 2$ (classical)

1.3 Q3: What is quantum superposition and how is it used?

Complete Answer:

Superposition allows quantum systems to exist in multiple states simultaneously until measured.

Mathematical Principle: Linear combinations of quantum states are valid states:

$$|\psi\rangle = c_1 |\psi_1\rangle + c_2 |\psi_2\rangle + \dots + c_n |\psi_n\rangle \quad (3)$$

where $\sum_i |c_i|^2 = 1$

Practical Usage in Algorithms:

1. Parallel Computation (Deutsch-Jozsa):

$$|\psi\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \quad (4)$$

Evaluates function on all inputs simultaneously.

2. Search (Grover): Creates uniform superposition, then amplifies target state amplitude through interference.

3. Optimization (QAOA): Explores solution space in parallel via superposition of all candidate solutions.

Code - Creating n-qubit Superposition:

```
def create_uniform_superposition(n_qubits):
    qc = QuantumCircuit(n_qubits)
    qc.h(range(n_qubits)) # Hadamard on all qubits
    # Creates |> = (1/ (2^n)) |x> for all x {0,1}^n
    return qc
```

Key Insight: Superposition alone doesn't give speedup - must combine with interference to amplify correct answers.

1.4 Q4: Explain the measurement problem in quantum mechanics

Complete Answer:

Measurement causes quantum state collapse from superposition to definite classical state.

Before Measurement: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$

After Measurement:

- Outcome 0 with probability $P(0) = |\alpha|^2$, state becomes $|0\rangle$
- Outcome 1 with probability $P(1) = |\beta|^2$, state becomes $|1\rangle$

Projection Measurement: For observable $\hat{A} = \sum_i a_i |a_i\rangle \langle a_i|$:

$$P(a_i) = \langle a_i | \psi \rangle \langle \psi | a_i \rangle = |\langle a_i | \psi \rangle|^2 \quad (5)$$

Expected value: $\langle A \rangle = \sum_i a_i P(a_i)$

Implications for Quantum Programming:

1. Cannot directly observe quantum states - only probabilities
2. Multiple shots needed - run circuit 1000-10000 times
3. Measurement destroys superposition - irreversible process
4. Statistical uncertainty: $\sigma = \sqrt{p(1-p)/n_{shots}}$
5. Mid-circuit measurements affect subsequent operations

Measuring in Different Bases:

```
# Computational (Z) basis - default
qc.measure(0, 0)

# X basis measurement
qc.h(0)
qc.measure(0, 0)
# Hadamard transforms X basis to Z basis

# Y basis measurement
qc.sdg(0) # S-dagger
qc.h(0)
qc.measure(0, 0)
```

Weak Measurement: Partial information extraction without full collapse (advanced topic).

1.5 Q5: What is quantum interference and how do algorithms exploit it?

Complete Answer:

Quantum interference occurs when probability amplitudes combine constructively or destructively.

Mathematical Principle: For two paths to same final state:

$$\text{Amplitude} = A_1 + A_2 \quad (6)$$

$$\text{Probability} = |A_1 + A_2|^2 = |A_1|^2 + |A_2|^2 + 2\text{Re}(A_1^* A_2) \quad (7)$$

The cross term $2\text{Re}(A_1^* A_2)$ is interference:

- **Constructive:** Same phase $\Rightarrow$ amplitudes add
- **Destructive:** Opposite phase $\Rightarrow$ amplitudes cancel

Algorithm Examples:

1. Deutsch-Jozsa:

- Balanced function: Destructive interference $\Rightarrow$ measure non-zero
- Constant function: Constructive interference $\Rightarrow$ measure zero

2. Grover's Algorithm:

- Oracle: Marks target with negative phase
- Diffusion: Inverts about average amplitude
- Combined effect: Amplifies target, suppresses others

```
# Grover iteration (simplified)
def grover_iteration(qc, oracle):
    # Oracle marks target
    qc.append(oracle, range(n))

    # Diffusion operator (inversion about average)
    qc.h(range(n))
    qc.x(range(n))
    qc.h(n-1)
    qc.mcx(list(range(n-1)), n-1) # Multi-controlled-X
    qc.h(n-1)
    qc.x(range(n))
    qc.h(range(n))
    # Repeating sqrt(N) times amplifies target to ~100%
```

3. Quantum Phase Estimation: Uses interference in Fourier space to extract eigenvalues.

Key Insight: Interference is why quantum algorithms can be more efficient - they eliminate wrong answers while reinforcing correct ones.

2 Quantum Gates and Circuits

2.1 Q6: Describe fundamental single-qubit gates

Complete Answer:

Pauli Gates:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (\text{bit flip}) \quad (8)$$

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \quad (\text{bit + phase flip}) \quad (9)$$

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad (\text{phase flip}) \quad (10)$$

Hadamard Gate:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad (11)$$

Creates superposition: $H|0\rangle = |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$

Phase Gates:

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} = \sqrt{Z} \quad (12)$$

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} = \sqrt{S} \quad (13)$$

Rotation Gates:

$$R_x(\theta) = e^{-i\theta X/2} = \begin{pmatrix} \cos \frac{\theta}{2} & -i \sin \frac{\theta}{2} \\ -i \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix} \quad (14)$$

$$R_y(\theta) = e^{-i\theta Y/2} = \begin{pmatrix} \cos \frac{\theta}{2} & -\sin \frac{\theta}{2} \\ \sin \frac{\theta}{2} & \cos \frac{\theta}{2} \end{pmatrix} \quad (15)$$

$$R_z(\phi) = e^{-i\phi Z/2} = \begin{pmatrix} e^{-i\phi/2} & 0 \\ 0 & e^{i\phi/2} \end{pmatrix} \quad (16)$$

Universal Decomposition: Any single-qubit unitary:

$$U = e^{i\alpha} R_z(\phi) R_y(\theta) R_z(\lambda) \quad (17)$$

Implementation:

```
# Qiskit examples
qc.x(0) # Pauli-X
qc.h(0) # Hadamard
qc.s(0) # S gate
qc.t(0) # T gate
qc.rx(np.pi/4, 0) # X-rotation
qc.ry(np.pi/3, 0) # Y-rotation
qc.rz(np.pi/2, 0) # Z-rotation

# General unitary (U3 gate)
qc.u(theta, phi, lambda, 0)
```

Gate Properties:

- All are unitary: $U^\dagger U = I$
- Pauli gates are self-inverse: $X^2 = Y^2 = Z^2 = I$
- $H^2 = I$ (Hadamard is self-inverse)
- Gates form groups under composition

2.2 Q7: Explain multi-qubit gates

Complete Answer:

CNOT (Controlled-NOT):

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad (18)$$

Action: Flips target qubit if control is $|1\rangle$

- $|00\rangle \rightarrow |00\rangle, |01\rangle \rightarrow |01\rangle$
- $|10\rangle \rightarrow |11\rangle, |11\rangle \rightarrow |10\rangle$

Controlled-Z:

$$CZ = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \quad (19)$$

Symmetric: $CZ_{01} = CZ_{10}$ (no distinction between control/target)

SWAP Gate:

$$\text{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (20)$$

Exchanges qubit states: $|\psi_0\rangle |\psi_1\rangle \rightarrow |\psi_1\rangle |\psi_0\rangle$

Toffoli (CCNOT): 3-qubit gate: Flips target if both controls are $|1\rangle$

Reversible classical AND: Essential for reversible computation

Decompositions:

```
# SWAP from 3 CNOTs
qc.cx(0, 1)
qc.cx(1, 0)
qc.cx(0, 1)

# CZ from CNOT + Hadamards
qc.h(1)
qc.cx(0, 1)
qc.h(1)

# Toffoli from CNOTs and single-qubit gates
qc.h(2)
qc.cx(1, 2)
qc.tdg(2)
qc.cx(0, 2)
qc.t(2)
qc.cx(1, 2)
qc.tdg(2)
qc.cx(0, 2)
qc.t(1)
qc.t(2)
qc.cx(0, 1)
qc.h(2)
qc.t(0)
qc.tdg(1)
qc.cx(0, 1)
```

Universality:

- CNOT + all single-qubit gates = universal gate set
- Can implement any quantum algorithm
- Toffoli + H = universal for quantum computation

Hardware Constraints:

- Not all qubit pairs can perform CNOT directly
- Requires routing via SWAP gates on limited connectivity
- Two-qubit gates have 10-100x higher error than single-qubit

2.3 Q8: How do you design and optimize quantum circuits?

Complete Answer:

Circuit Design Principles:

1. Minimize Depth: Depth = number of sequential gate layers

- Lower depth $\Rightarrow$ less decoherence
- Parallelizable gates execute simultaneously
- Critical for NISQ devices

2. Minimize Two-Qubit Gates:

- CNOT error rate: 0.5-2% (vs 0.01-0.1% for single-qubit)
- CNOT count is primary optimization target
- Use gate cancellation and commutation

3. Match Hardware Topology:

```
from qiskit import transpile
from qiskit.providers.fake_provider import FakeMelbourne

# Get backend topology
backend = FakeMelbourne()
coupling_map = backend.configuration().coupling_map

# Transpile respecting connectivity
transpiled_qc = transpile(
    qc,
```

```

    backend=backend,
    optimization_level=3, # Heavy optimization
    initial_layout=None # Auto-select best qubit mapping
)

print(f"Original depth: {qc.depth()}")
print(f"Transpiled depth: {transpiled_qc.depth()}")
print(f"Original CNOT count: {qc.count_ops().get('cx', 0)}")
print(f"Transpiled CNOT count: {transpiled_qc.count_ops().get('cx', 0)}")

```

Optimization Techniques:

Gate Cancellation:

- $H \cdot H = I$, $X \cdot X = I$, $Z \cdot Z = I$
- Adjacent inverses cancel

Gate Commutation:

- Reorder commuting gates: $[Z_i, Z_j] = 0$
- Enable more cancellations
- Z commutes with CNOT control, X with CNOT target

Gate Fusion: Combine consecutive single-qubit rotations:

$$R_z(\phi_2)R_y(\theta_2)R_z(\phi_1) = R_z(\phi')R_y(\theta')R_z(\lambda') \quad (21)$$

Custom Optimization Pass:

```

from qiskit.transpiler import PassManager
from qiskit.transpiler.passes import *

# Build custom optimization pipeline
pm = PassManager()
pm.append(Unroller(['u', 'cx'])) # Decompose to basis gates
pm.append(Optimize1qGates()) # Fuse single-qubit gates
pm.append(CommutativeCancellation()) # Use commutation
pm.append(CXCancellation()) # Cancel adjacent CNOTs
pm.append(Optimize1qGatesDecomposition()) # Final cleanup

optimized_qc = pm.run(qc)

```

Circuit Metrics:

```

def analyze_circuit(qc):
    ops = qc.count_ops()
    return {
        'depth': qc.depth(),
        'size': qc.size(),
        'width': qc.num_qubits,
        'cx_count': ops.get('cx', 0),
        'single_qubit_gates': qc.size() - ops.get('cx', 0),
        'estimated_fidelity': (0.999 ** ops.get('u', 0)) *
                               (0.99 ** ops.get('cx', 0))
    }

```

2.4 Q9: Explain quantum circuit transpilation

Complete Answer:

Transpilation converts abstract circuit to hardware-executable form.

Transpilation Stages:

1. **Gate Decomposition:** Convert to native gate set (e.g., $\{U3, CNOT\}$ for IBM)

```

# Custom gate decomposed to native gates
qc.swap(0, 1)
# Becomes: CNOT(0,1), CNOT(1,0), CNOT(0,1)

```

2. **Qubit Mapping:** Map logical qubits to physical qubits considering:

- Coupling map (connectivity)
- Qubit quality (T1, T2, gate fidelities)
- Readout fidelities

```
# Specify initial layout
from qiskit.transpiler import CouplingMap
coupling = CouplingMap([[0,1], [1,2], [2,3]])
initial_layout = [2, 3, 1] # Map logical [0,1,2] to physical [2,3,1]

transpiled = transpile(
    qc,
    coupling_map=coupling,
    initial_layout=initial_layout
)
```

3. Routing: Insert SWAP gates for non-adjacent operations:

- Routing algorithms: SABRE, basic, stochastic, lookahead
- Trade-off: SWAPs vs circuit depth
- Each SWAP = 3 CNOTs

4. Optimization: Multiple levels available:

```
# Level 0: No optimization (fast)
t0 = transpile(qc, backend, optimization_level=0)

# Level 1: Light (gate cancellation, commutation)
t1 = transpile(qc, backend, optimization_level=1)

# Level 2: Medium (+ routing optimization)
t2 = transpile(qc, backend, optimization_level=2)

# Level 3: Heavy (+ noise-aware, layout search)
t3 = transpile(qc, backend, optimization_level=3)
```

5. Scheduling (Optional): Assign gate start times considering:

- Gate durations
- Dynamical decoupling
- Parallelization opportunities

Transpilation Comparison:

```
def compare_transpilation_levels(qc, backend):
    results = {}
    for level in range(4):
        tc = transpile(qc, backend, optimization_level=level)
        results[f'level_{level}'] = {
            'depth': tc.depth(),
            'cx_count': tc.count_ops().get('cx', 0),
            'size': tc.size()
        }
    return results

# Example output:
# level_0: depth=100, cx=50
# level_1: depth=85, cx=45
# level_2: depth=75, cx=42
# level_3: depth=65, cx=38
```

Advanced: Pulse-Level:

```
from qiskit import pulse

# For backends supporting pulse schedules
with pulse.build(backend) as my_program:
    pulse.play(GaussianSquare(...), DriveChannel(0))
    pulse.call(cx_sched) # Custom calibrated CNOT
    # Fine-grained control over microwave pulses
```

2.5 Q10: What are limitations of NISQ devices?

Complete Answer:

NISQ (Noisy Intermediate-Scale Quantum) devices have 50-1000 qubits but significant noise.

Key Limitations:

1. Decoherence Times:

- T_1 (relaxation): 10-100 μs (superconducting)
- T_2 (dephasing): 20-200 μs

- Limits circuit execution time
- Depth limit: 100-1000 gates

2. Gate Errors:

- Single-qubit: 0.01-0.1% error
- Two-qubit (CNOT): 0.5-2% error
- Measurement: 1-5% error

Error accumulation for n gates with error ϵ :

$$F_{\text{circuit}} \approx e^{-n\epsilon} \approx (1 - \epsilon)^n \quad (22)$$

Example: 100 CNOTs at 1% error $\Rightarrow$ 37% fidelity

3. Limited Connectivity:

- Not all qubit pairs can interact directly
- Requires routing via SWAP gates
- Increases depth and error

4. No Error Correction:

- Full QEC needs 1000 physical qubits per logical
- Current devices cannot afford this overhead
- Must use error mitigation instead

5. Crosstalk:

- Gates on one qubit affect neighbors
- Simultaneous gates interfere
- Limits parallelization

6. Readout Errors:

- Measurement outcomes incorrect 1-5%
- Can bias algorithmic results
- Requires calibration matrices

Strategies for NISQ Era:

Error Mitigation:

```
from qiskit.ignis.mitigation import CompleteMeasFitter

# Calibration circuits
cal_circuits, state_labels = complete_meas_cal(qr=qc.qregs[0])
cal_results = execute(cal_circuits, backend, shots=5000).result()

# Build mitigation filter
meas_fitter = CompleteMeasFitter(cal_results, state_labels)
meas_filter = meas_fitter.filter

# Apply to results
noisy_counts = execute(qc, backend, shots=1000).result().get_counts()
mitigated_counts = meas_filter.apply(noisy_counts)
```

Variational Algorithms:

- VQE, QAOA: Short circuits + classical optimization
- Noise-resilient by design
- Most promising for near-term applications

Circuit Optimization:

- Minimize depth and CNOT count

- Use hardware-efficient ansätze
- Exploit problem structure

Performance Metrics:

```
def nisq_capability_score(backend_properties):
    # Simplified metric
    avg_t1 = np.mean([backend_properties.t1(i)
                      for i in range(backend_properties.num_qubits)])
    avg_cx_error = np.mean([props['gate_error'][0]
                           for gate, props in backend_properties.gates
                           if gate == 'cx'])

    # Higher is better
    score = (avg_t1 * 1e6) / avg_cx_error
    return score
```

3 Quantum Algorithms

3.1 Q11: Explain Grover's search algorithm in detail

Complete Answer:

Grover's algorithm provides quadratic speedup for unstructured search.

Problem: Search unsorted database of N items, find marked item in $O(\sqrt{N})$ queries.

Algorithm Steps:

1. **Initialize:** Create uniform superposition

$$|s\rangle = H^{\otimes n} |0\rangle^n = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle \quad (23)$$

2. **Oracle:** Mark target with phase flip

$$O|x\rangle = (-1)^{f(x)} |x\rangle = \begin{cases} -|x\rangle & \text{if } x = \omega \\ |x\rangle & \text{otherwise} \end{cases} \quad (24)$$

3. **Diffusion:** Inversion about average

$$D = 2|s\rangle\langle s| - I \quad (25)$$

4. **Iterate:** Repeat $O \circ D$ approximately $\frac{\pi}{4}\sqrt{N}$ times

Geometric Picture:

- State space spanned by $|s\rangle$ (superposition) and $|\omega\rangle$ (target)
- Each iteration rotates by $\theta \approx 2/\sqrt{N}$ toward $|\omega\rangle$
- After $\sim \frac{\pi}{4}\sqrt{N}$ iterations: overlap with $|\omega\rangle \approx 1$

Complete Implementation:

```
from qiskit import QuantumCircuit
import numpy as np

def grover_oracle(marked_states, n):
    """Oracle marks target states with phase flip"""
    qc = QuantumCircuit(n)

    for target in marked_states:
        # Convert target to binary string
        target_bits = format(target, f'0{n}b')

        # Flip qubits that are 0 in target
        for i, bit in enumerate(target_bits):
            if bit == '0':
                qc.x(i)

        # Multi-controlled Z (phase flip when all qubits are 1)
        if n == 1:
            qc.z(0)
        elif n == 2:
            qc.cz(0, 1)
        else:
            qc.h(n-1)
            qc.mcx(list(range(n-1)), n-1) # Multi-controlled-X
            qc.h(n-1)
```

```

        # Flip back
        for i, bit in enumerate(target_bits):
            if bit == '0':
                qc.x(i)

    return qc

def grover_diffusion(n):
    """Diffusion operator D = 2|s><s| - I"""
    qc = QuantumCircuit(n)

    # Transform to computational basis
    qc.h(range(n))
    qc.x(range(n))

    # Multi-controlled Z
    qc.h(n-1)
    if n > 2:
        qc.mcx(list(range(n-1)), n-1)
    elif n == 2:
        qc.cx(0, 1)
    else:
        qc.x(0)
    qc.h(n-1)

    # Transform back
    qc.x(range(n))
    qc.h(range(n))

    return qc

def grover_algorithm(marked_states, n_qubits):
    """Full Grover's algorithm"""
    N = 2**n_qubits
    n_iterations = int(np.pi/4 * np.sqrt(N / len(marked_states)))

    qc = QuantumCircuit(n_qubits, n_qubits)

    # Initialize superposition
    qc.h(range(n_qubits))

    # Grover iterations
    for _ in range(n_iterations):
        # Oracle
        oracle = grover_oracle(marked_states, n_qubits)
        qc.compose(oracle, inplace=True)

        # Diffusion
        diffusion = grover_diffusion(n_qubits)
        qc.compose(diffusion, inplace=True)

    # Measure
    qc.measure(range(n_qubits), range(n_qubits))

    return qc

# Example: Search for state |101> in 8-item database
qc = grover_algorithm(marked_states=[5], n_qubits=3)
# After 2 iterations, measures |101> with ~100% probability

```

Success Probability:

$$P_{\text{success}}(k) = \sin^2 \left((2k + 1) \arcsin \frac{1}{\sqrt{N}} \right) \quad (26)$$

where k is number of iterations.

Complexity:

- Oracle calls: $O(\sqrt{N})$ vs classical $O(N)$
- Gate count per iteration: $O(n)$
- Total: $O(n\sqrt{N})$ gates

Applications:

- Database search
- SAT solving
- Constraint satisfaction
- Collision finding (cryptography)

Multiple Solutions: If M marked items among N total:

$$\text{Iterations} = \frac{\pi}{4} \sqrt{\frac{N}{M}} \quad (27)$$

3.2 Q12: Explain the Quantum Fourier Transform

Complete Answer:

The QFT is quantum analogue of discrete Fourier transform, fundamental to many algorithms.

Definition:

$$\text{QFT} |j\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i j k / 2^n} |k\rangle \quad (28)$$

Matrix Form (n=3):

$$\text{QFT} = \frac{1}{\sqrt{8}} \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \omega & \omega^2 & \omega^3 & \omega^4 & \omega^5 & \omega^6 & \omega^7 \\ \vdots & & & & & & & \vdots \end{pmatrix} \quad (29)$$

where $\omega = e^{2\pi i/8}$

Product Representation:

$$\text{QFT} |j_1 \dots j_n\rangle = \frac{1}{\sqrt{2^n}} \bigotimes_{k=1}^n (|0\rangle + e^{2\pi i (0.j_k j_{k+1} \dots j_n)} |1\rangle) \quad (30)$$

Binary fraction: $0.j_k j_{k+1} \dots j_n = \frac{j_k}{2} + \frac{j_{k+1}}{4} + \dots + \frac{j_n}{2^{n-k+1}}$

Circuit Implementation:

```
def qft(n):
    """n-qubit QFT"""
    qc = QuantumCircuit(n)

    for j in range(n):
        # Hadamard on qubit j
        qc.h(j)

        # Controlled rotations
        for k in range(j+1, n):
            angle = 2 * np.pi / (2**(k-j+1))
            qc.cp(angle, k, j) # Controlled-phase

    # Reverse qubit order (swap)
    for i in range(n//2):
        qc.swap(i, n-i-1)

    return qc

def inverse_qft(n):
    """Inverse QFT"""
    return qft(n).inverse()
```

Gate Complexity:

- Hadamards: n
- Controlled-phase: $\frac{n(n-1)}{2}$
- SWAPs: $\lfloor n/2 \rfloor$
- Total: $O(n^2)$ gates

Compare to classical FFT: $O(N \log N) = O(2^n \cdot n)$ Exponential speedup in gate count!

Applications:

1. **Phase Estimation:** Estimate eigenvalue λ of unitary U :

```
def quantum_phase_estimation(unitary, n_count, eigenstate):
    """Estimate phase of eigenstate"""
    qc = QuantumCircuit(n_count + unitary.num_qubits, n_count)

    # Initialize counting qubits to |+>
    qc.h(range(n_count))

    # Prepare eigenstate
    qc.initialize(eigenstate, range(n_count, qc.num_qubits))

    # Controlled unitaries U^(2^j)
    for j in range(n_count):
        power = 2**(n_count - j - 1)
        controlled_u = unitary.power(power).control(1)
        qc.append(controlled_u, [j] + list(range(n_count, qc.num_qubits)))

    # Inverse QFT on counting register
    qc.append(inverse_qft(n_count), range(n_count))
```

```
# Measure
qc.measure(range(n_count), range(n_count))

return qc

# Result in binary gives phase:      = measured_value / 2^n_count
# Eigenvalue:      = e^(2 i )
```

2. Shor's Algorithm: Period finding via QFT of modular exponentiation

3. HHL Algorithm: Solving linear systems $Ax = b$

4. Quantum Simulation: Time evolution via Trotter decomposition + QFT

Approximate QFT: For hardware efficiency, drop small-angle rotations:

```
def approximate_qft(n, approximation_degree=5):
    """QFT keeping only rotations > 2^-approximation_degree"""
    qc = QuantumCircuit(n)

    for j in range(n):
        qc.h(j)
        for k in range(j+1, min(j+approximation_degree, n)):
            angle = 2 * np.pi / (2**(k-j+1))
            qc.cp(angle, k, j)

    for i in range(n//2):
        qc.swap(i, n-i-1)

    return qc
# Reduces gate count while maintaining accuracy
```

3.3 Q13: Describe variational quantum algorithms (VQE, QAOA)

Complete Answer:

Variational algorithms are hybrid quantum-classical methods suitable for NISQ devices.

General Framework:

1. Parameterized Circuit (Ansatz):

$$|\psi(\theta)\rangle = U(\theta) |0\rangle \quad (31)$$

2. Cost Function:

$$C(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle \quad (32)$$

3. Classical Optimization:

$$\theta^* = \arg \min_{\theta} C(\theta) \quad (33)$$

4. Iterate until convergence

VQE (Variational Quantum Eigensolver):

Find ground state energy of Hamiltonian.

Ansatz Design:

```
def hardware_efficient_ansatz(n_qubits, depth):
    """Hardware-efficient ansatz"""
    from qiskit.circuit import Parameter

    qc = QuantumCircuit(n_qubits)
    params = []

    for d in range(depth):
        # Rotation layer
        for i in range(n_qubits):
            theta = Parameter(f' {d}_{i}')
            qc.ry(theta, i)
            params.append(theta)

        # Entanglement layer
        for i in range(n_qubits-1):
            qc.cx(i, i+1)

    return qc, params

# UCCSD ansatz for chemistry
from qiskit_nature.circuit.library import UCCSD

def uccsd_ansatz(num_orbitals, num_particles):
    """Chemistry-inspired ansatz"""
    ansatz = UCCSD(
        num_spatial_orbitals=num_orbitals,
        num_particles=num_particles,
        qubit_converter=JordanWignerMapper()
    )
    return ansatz
```

Complete VQE:

```

from qiskit.algorithms import VQE
from qiskit.algorithms.optimizers import SLSQP, COBYLA
from qiskit.opflow import Z, X, I

# Define Hamiltonian (H2 molecule example)
H = (-1.05 * (I^I)) + (0.39 * (Z^I)) + \
    (-0.39 * (I^Z)) + (-0.01 * (Z^Z)) + (0.18 * (X^X))

# Create ansatz
ansatz, params = hardware_efficient_ansatz(n_qubits=2, depth=2)

# VQE
vqe = VQE(
    ansatz=ansatz,
    optimizer=SLSQP(maxiter=1000),
    quantum_instance=backend
)

result = vqe.compute_minimum_eigenvalue(H)

print(f"Ground state energy: {result.eigenvalue.real:.6f}")
print(f"Optimal parameters: {result.optimal_point}")
print(f"Optimizer evaluations: {result.optimizer_evals}")

```

Gradient Computation (Parameter Shift):

$$\frac{\partial C}{\partial \theta_i} = \frac{C(\theta + \pi/2e_i) - C(\theta - \pi/2e_i)}{2} \quad (34)$$

```

def parameter_shift_gradient(qc, hamiltonian, params, param_idx):
    """Compute gradient using parameter shift rule"""
    shift = np.pi / 2

    params_plus = params.copy()
    params_plus[param_idx] += shift
    E_plus = evaluate_expectation(qc, hamiltonian, params_plus)

    params_minus = params.copy()
    params_minus[param_idx] -= shift
    E_minus = evaluate_expectation(qc, hamiltonian, params_minus)

    return (E_plus - E_minus) / 2

```

QAOA (Quantum Approximate Optimization):

For combinatorial optimization problems.

Problem Hamiltonian:

$$H_C = \sum_{\alpha} C_{\alpha}(z) \quad (35)$$

QAOA Circuit:

$$|\psi(\gamma, \beta)\rangle = U_B(\beta_p)U_C(\gamma_p) \cdots U_B(\beta_1)U_C(\gamma_1)|+\rangle^{\otimes n} \quad (36)$$

where:

$$U_C(\gamma) = e^{-i\gamma H_C} \quad (\text{problem unitary}) \quad (37)$$

$$U_B(\beta) = e^{-i\beta \sum_i X_i} \quad (\text{mixer}) \quad (38)$$

MaxCut Example:

```

from qiskit_optimization.applications import Maxcut
from qiskit.algorithms import QAOA
import networkx as nx

# Define graph
G = nx.Graph()
G.add_edges_from([(0,1), (1,2), (2,3), (3,0), (0,2)])

# MaxCut problem
maxcut = Maxcut(G)
qp = maxcut.to_quadratic_program()
operator, offset = qp.to_ising()

# QAOA
qaoa = QAOA(
    optimizer=COBYLA(maxiter=200),
    reps=3, # p layers
    quantum_instance=backend
)

result = qaoa.compute_minimum_eigenvalue(operator)

# Extract solution

```

```
partition = maxcut.sample_most_likely(result.eigenstate)
cut_value = maxcut.get_graph_solution(partition)

print(f"Optimal cut: {cut_value}")
print(f"Partition: {partition}")
```

Performance:

- Approximation ratio improves with p (number of layers)
- For MaxCut on 3-regular graphs: $p = 1$ gives ≥ 0.692 approximation
- As $p \rightarrow \infty$, approaches optimal solution

Challenges:

- **Barren plateaus:** Gradients vanish exponentially with depth
- **Local minima:** Optimization landscape is non-convex
- **Shot noise:** Finite sampling introduces statistical errors

Mitigation Strategies:

```
# Layerwise training
def layerwise_vqe(hamiltonian, max_depth):
    params = []
    for depth in range(1, max_depth+1):
        ansatz, new_params = hardware_efficient_ansatz(n_qubits, depth)

        # Initialize new layer parameters only
        init_params = np.concatenate([
            params, # Keep previous layers
            np.random.rand(len(new_params) - len(params)) * 0.01 # Small random for new layer
        ])

        vqe = VQE(ansatz, SLSQP(), qi)
        result = vqe.compute_minimum_eigenvalue(hamiltonian)
        params = result.optimal_point

    return params

# Warm starting from classical solution
def warm_start_qaoa(problem, classical_solution):
    # Map classical solution to initial parameters
    initial_gamma = encode_solution(classical_solution)
    initial_beta = np.ones(p) * 0.1

    qaoa = QAOA(reps=p, initial_point=np.concatenate([initial_gamma, initial_beta]))
    return qaoa.compute_minimum_eigenvalue(problem)
```